

UNIT 2: PROBLEM SOLVING, SEARCH AND KNOWLEDGE REPRESENTATION

Course Pattern: TE IT - 2024 Pattern (SPPU)
Status: Publication-Grade Theory Guide

Academic Term: Semester V (2026)
Course Code: PCC-302-IT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 2.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
Problem Formulation & State Space	5-component problem definition ($s_0, A(s), \text{Result}, G(s), c$). State space graph vs search tree. Worked 8-puzzle formulation.	High Priority (6-8 Marks Qs)
Uninformed Search Strategies	Breadth First Search (BFS), Depth First Search (DFS), Uniform Cost Search (UCS). Time, space, completeness, optimality comparison.	High Priority (8-10 Marks Qs)
Informed Search (Heuristic)	Heuristic functions $h(n)$, Greedy Best First Search, A* Search ($f(n) = g(n) + h(n)$). Admissibility & Consistency conditions.	High Priority (8-10 Marks Qs)
Local Search & Hill Climbing	Hill Climbing algorithm, Local Maxima, Ridges, Plateaux pitfalls. Simulated Annealing, Local Beam Search.	High Priority (6-8 Marks Qs)
Knowledge Representation & Logic	Knowledge-Based Agents (Tell, Ask). Propositional Logic syntax, semantics, truth tables, equivalences, Modus Ponens.	High Priority (7-9 Marks Qs)
First-Order Logic (FOL)	Predicate Logic syntax, quantifiers ($\forall, \exists$), WFFs. Translating English sentences into FOL.	High Priority (6-8 Marks Qs)
Inference Chaining & Expert Systems	Horn clauses. Forward Chaining vs Backward Chaining algorithms. Expert System Architecture & Components.	High Priority (8-10 Marks Qs)

Problem Formulation and State Space Representation

Problem-Solving Agents and Problem Formulation

A **Problem-Solving Agent** is a goal-based agent that decides what to do by finding sequences of actions that lead to desirable goal states. Problem solving involves four fundamental steps: **Goal Formulation**, **Problem Formulation**, **Search**, and **Execution**.

A formal problem is defined by **five core components**:

- Initial State (s_0):** The starting state of the agent in the environment (e.g., $\text{In}(\text{Arad})$).
- Actions ($A(s)$):** The set of valid actions available to the agent when in state s :

$$A(s) = \{a \mid a \text{ is a valid action in state } s\}$$

3. **Transition Model** ($\text{Result}(s, a)$): A formal function describing what state results from taking action a in state s :

$$s' = \text{Result}(s, a)$$

Together, the initial state, actions, and transition model define the **State Space** of the problem—the set of all states reachable from the initial state by any sequence of actions.

4. **Goal Test** ($G(s)$): A boolean test that determines whether a given state s is a goal state (e.g., $G(s) = [s == \text{In}(\text{Bucharest})]$).
5. **Path Cost** ($c(s, a, s')$): A numerical cost function assigning a cost to taking action a to transition from state s to s' . The step cost is denoted by $c(s, a, s')$. The path cost $g(n)$ is the sum of step costs along a path from the initial state to node n .

State Space Graph vs. Search Tree

- **State Space Graph:** A directed graph where nodes represent physical environment states and directed edges represent valid action transitions. The state space graph is finite if the number of environment states is finite.
- **Search Tree:** An explicit tree structure constructed dynamically by the search algorithm during execution. Root node corresponds to the initial state; branches represent action choices. The same physical state can appear in multiple nodes of a search tree due to different paths.

EXAMPLE:

Problem Formulation: The 8-Puzzle Game

- **States:** Any placement of 8 numbered tiles (1 to 8) and 1 blank space on a 3×3 grid. Total states $= \frac{9!}{2} = 181,440$.
- **Initial State:** A specified configuration of tiles (e.g., $[1, 2, 3; 4, 0, 5; 7, 8, 6]$ where 0 is blank).
- **Actions:** Moving the blank space {Left, Right, Up, Down}.
- **Transition Model:** Returns the resulting grid configuration after swapping the blank tile with the adjacent tile.
- **Goal Test:** Checks if tiles match the target ordered grid $[1, 2, 3; 4, 5, 6; 7, 8, 0]$.
- **Path Cost:** Each step costs 1; total path cost is the length of the action sequence.

Source: Russell & Norvig 4th Ed, Ch 3, p. 81-90

Search Strategies and Evaluation Criteria

Measuring Problem-Solving Performance

Search algorithms explore the search space by evaluating frontier nodes. They are formally evaluated across **four standard criteria**:

1. **Completeness:** Is the algorithm guaranteed to find a solution when one exists?
2. **Time Complexity:** How long (number of generated/expanded nodes) does it take to find a solution?
3. **Space Complexity:** How much memory is required to perform the search (maximum size of the frontier)?
4. **Optimality:** Does the strategy find the optimal solution with the lowest path cost C^* among all possible solutions?

Complexity Parameters

Complexity is measured using three fundamental search space parameters:

- b : **Branching Factor** — Maximum number of successors (children) of any node.
- d : **Depth of Shallowest Goal Node** — Number of steps from root to the nearest goal state.
- m : **Maximum Depth** of the state space (can be ∞ if loops exist).

Uninformed Search Strategies

Uninformed (Blind) search strategies have no domain-specific knowledge about the estimated cost to reach the goal state. They can only generate successors and distinguish goal states from non-goal states.

1. Breadth-First Search (BFS)

BFS expands the root node first, then expands all successors of the root, then all their successors, expanding the shallowest unexpanded node first.

- **Frontier Data Structure:** First-In, First-Out (**FIFO Queue**).
- **Completeness:** Yes (if branching factor b is finite).
- **Time Complexity:** $O(b^d)$
- **Space Complexity:** $O(b^d)$ (All nodes in memory at depth d).
- **Optimality:** Yes, if all step costs are equal (or unit cost $c = 1$). Non-optimal if step costs vary.

2. Depth-First Search (DFS)

DFS always expands the deepest node in the current frontier of the search tree.

- **Frontier Data Structure:** Last-In, First-Out (**LIFO Stack**).
- **Completeness:** No (Fails in infinite-depth spaces or spaces with loops without cycle checking).
- **Time Complexity:** $O(b^m)$ (Terrible if m is much larger than d).
- **Space Complexity:** $O(bm)$ (Linear space! Stores only the path from root to current node plus remaining unexpanded siblings).
- **Optimality:** No (May find a deep goal on the left branch before exploring a shallow goal on the right).

3. Uniform Cost Search (UCS / Dijkstra's Search)

UCS expands the node n with the lowest path cost $g(n)$ from the root node.

- **Frontier Data Structure:** **Priority Queue** ordered by path cost $g(n)$.
- **Goal Test Location:** Goal test is applied to a node **when it is selected for expansion**, not when it is generated.
- **Completeness:** Yes, if step costs are strictly positive ($\epsilon > 0$).
- **Time & Space Complexity:** $O(b^{1+\lceil \frac{C^*}{\epsilon} \rceil})$ where C^* is optimal path cost and ϵ is minimum step cost.
- **Optimality:** Yes (Guaranteed optimal for any non-negative step costs).

Comparative Evaluation Matrix of Uninformed Search

Algorithm	Frontier	Time	Space	Optimal?
Breadth-First Search (BFS)	FIFO Queue	$O(b^d)$	$O(b^d)$	Yes (Unit cost)
Depth-First Search (DFS)	LIFO Stack	$O(b^m)$	$O(bm)$	No
Uniform Cost Search (UCS)	Priority Queue (g)	$O(b^{1+\lceil \frac{C^*}{\epsilon} \rceil})$	$O(b^{1+\lceil \frac{C^*}{\epsilon} \rceil})$	Yes

Source: Russell & Norvig 4th Ed, Ch 3, p. 94-102

Informed (Heuristic) Search Strategies

Informed search strategies use domain-specific hints via a **Heuristic Function** $h(n)$ to guide the search towards goal states more efficiently:

$h(n)$ = Estimated cost of the cheapest path from node n to a goal state.

1. Greedy Best-First Search

Greedy best-first search expands the node that appears closest to the goal, evaluating nodes using $f(n) = h(n)$.

- **Frontier:** Priority Queue ordered by $h(n)$.
- **Completeness & Optimality:** Incomplete (can get stuck in loops) and Non-Optimal.

2. A* Search ($f(n) = g(n) + h(n)$)

A* Search evaluates nodes by combining the actual cost to reach the node $g(n)$ and the estimated cost to reach the goal $h(n)$:

$$f(n) = g(n) + h(n)$$

where $f(n)$ is the estimated cost of the cheapest solution passing through node n .

IMPORTANT:

Conditions for A* Optimality:

1. **Admissibility (Tree Search Optimality):** A heuristic $h(n)$ is **admissible** if it **never overestimates** the true cost to reach the goal:

$$h(n) \leq h^*(n) \quad \forall n$$

where $h^*(n)$ is the true optimal cost from n to goal. An admissible heuristic is optimistic.

2. **Consistency / Monotonicity (Graph Search Optimality):** A heuristic $h(n)$ is **consistent** if for every node n and every successor n' generated by action a :

$$h(n) \leq c(n, a, n') + h(n')$$

Consistency ensures that $f(n)$ values along any path are non-decreasing ($f(n') \geq f(n)$). Every consistent heuristic is admissible.

Source: Russell & Norvig 4th Ed, Ch 3, p. 102-120

Local Search and Hill Climbing

Optimization Problem Formulation

In many problems, the path to the goal is irrelevant; only the **final goal state itself** matters (e.g., 8-Queens problem, IC chip layout, TSP). **Local Search Algorithms** operate using a single current node and move only to neighbors, using memory $O(1)$.

The state space is viewed as a **State-Space Landscape** with an objective value / elevation function.

Hill-Climbing Search (Greedy Local Search)

Hill climbing is a loop that continuously moves in the direction of increasing value (steepest ascent). It terminates when it reaches a peak where no neighbor has a higher value.

WARNING:**3 Major Pitfalls of Hill-Climbing:**

1. **Local Maxima:** A peak that is higher than all its neighboring states but lower than the global maximum. The algorithm gets stuck because all moves lead downward.
 - *Remedy:* **Random-Restart Hill Climbing** (conducts multiple searches from randomly generated initial states).
2. **Plateaux / Flat Maxima:** A flat area of the state-space landscape where neighboring states have the exact same value. The algorithm performs a random walk.
 - *Remedy:* Allow **Sideways Moves** (limit maximum consecutive flat steps).
3. **Ridges:** A sequence of local maxima where the grid moves are aligned diagonally to the slope. Moving in any single cardinal direction results in a drop.
 - *Remedy:* Move in multiple directions simultaneously or use joint action moves.

Simulated Annealing & Local Beam Search

- **Simulated Annealing:** Combines hill climbing with a random walk to yield efficiency and completeness. Instead of choosing the best move, it picks a random move. If the move improves the state ($\Delta E > 0$), it is accepted. If not, it accepts the bad move with probability:

$$P(\text{accept bad move}) = e^{\Delta \frac{E}{T}}$$

where T is a temperature parameter that decreases according to a cooling schedule.

- **Local Beam Search:** Keeps track of k states rather than just one. At each step, all successors of all k states are generated, and the top k overall successors are selected.

Source: Russell & Norvig 4th Ed, Ch 4, p. 131-140

Knowledge Representation and Knowledge-Based Agents**Knowledge-Based Agents**

A **Knowledge-Based (KB) Agent** maintains an internal **Knowledge Base (KB)** containing formal sentences representing assertions about the world.

- Key Operations:
 - $\text{Tell}(\text{KB}, \text{sentence})$: Adds a new assertion to the KB.
 - $\text{Ask}(\text{KB}, \text{query})$: Queries the KB to infer answers based on logical reasoning.

Requirements of a Knowledge Representation Language

1. **Representational Adequacy:** Ability to represent all kinds of knowledge required in the domain.
2. **Inferential Adequacy:** Ability to manipulate representation structures to derive new knowledge from existing facts.
3. **Inferential Efficiency:** Ability to direct inferential processes into productive paths using heuristics.
4. **Acquisition Efficiency:** Ability to acquire new knowledge easily and integrate it cleanly.

Propositional Logic**Syntax and Semantics**

Propositional Logic operates on proposition symbols (P, Q, R) representing facts that can be True (T) or False (F).

Logical Connectives:

- Negation ($\neg P$): NOT P
- Conjunction ($P \wedge Q$): P AND Q
- Disjunction ($P \vee Q$): P OR Q
- Implication ($P \Rightarrow Q$): If P then Q (equivalent to $\neg P \vee Q$)
- Biconditional ($P \Leftrightarrow Q$): P if and only if Q

Truth Tables and Logical Equivalences

P	Q	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
T	T	T	T	T	T
T	F	F	T	F	F
F	T	F	T	T	F
F	F	F	F	T	T

NOTE:**Key Logical Equivalences:**

- **De Morgan's Laws:** $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ and $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
- **Implication Elimination:** $P \Rightarrow Q \equiv \neg P \vee Q$
- **Contrapositive Law:** $P \Rightarrow Q \equiv \neg Q \Rightarrow \neg P$

Inference Rules

- **Modus Ponens:** Given $P \Rightarrow Q$ and P , infer Q .
- **Modus Tollens:** Given $P \Rightarrow Q$ and $\neg Q$, infer $\neg P$.
- **Resolution:** Given $A \vee B$ and $\neg B \vee C$, infer $A \vee C$.

Source: Russell & Norvig 4th Ed, Ch 7, p. 236-260

Predicate Logic (First-Order Logic - FOL)**Limitations of Propositional Logic**

Propositional logic lacks expressive power to state general rules efficiently (e.g., "All humans are mortal"). It treats sentences as atomic blocks without internal structure.

Syntax and Quantifiers of FOL

First-Order Logic represents the world in terms of **Objects**, **Predicates (Relations)**, and **Functions**.

Quantifiers:

1. **Universal Quantifier ($\forall x$):** "For all x ". Expresses properties true for all entities in domain. Usually paired with implication ($\Rightarrow$).

$$\forall x, (\text{Human}(x) \Rightarrow \text{Mortal}(x))$$

2. **Existential Quantifier ($\exists x$):** "There exists an x ". Expresses properties true for at least one entity. Usually paired with conjunction ($\wedge$).

$$\exists x, (\text{Student}(x) \wedge \text{Pass}(x))$$

EXAMPLE:

Translating Natural Language to FOL:

- “Every doctor has a stethoscope.”
 $\forall x, (\text{Doctor}(x) \Rightarrow \exists y, (\text{Stethoscope}(y) \wedge \text{Has}(x, y)))$
- “Someone loves everyone.”
 $\exists x, \forall y, \text{Loves}(x, y)$

Source: Russell & Norvig 4th Ed, Ch 8, p. 280-305

Forward Chaining and Backward Chaining

Inference algorithms operate on **Horn Clauses**—disjunctions of literals of which at most one is positive (Definite Clauses: $P_1 \wedge P_2 \wedge \dots \wedge P_k \Rightarrow Q$).

1. Forward Chaining (Data-Driven Reasoning)

- **Mechanism:** Starts from known facts in the Knowledge Base and fires rules whose premises are satisfied, adding new conclusions to the KB. Repeats until the query goal is derived or no new facts can be added.
- **Direction:** Bottom-up (Data $\rightarrow$ Goal).
- **Use Cases:** Real-time monitoring, diagnostic processing, automated control systems.

2. Backward Chaining (Goal-Driven Reasoning)

- **Mechanism:** Starts from the query goal statement and searches backward through rules to find supporting premises. If a premise is unknown, it becomes a sub-goal to prove.
- **Direction:** Top-down (Goal $\rightarrow$ Facts).
- **Use Cases:** Expert diagnostic systems, interactive query tools, PROLOG execution engine.

Comparative Matrix: Forward vs. Backward Chaining

Parameter	Forward Chaining	Backward Chaining
Reasoning Approach	Data-Driven (Bottom-Up)	Goal-Driven (Top-Down)
Starting Point	Initial known facts	Goal/Hypothesis query
Rule Triggering	Fires all satisfied rules	Selects rules leading to goal
Efficiency	May derive many irrelevant facts	Highly targeted; focuses only on goal
Primary Application	Monitoring, control systems, synthesis	Diagnosis, troubleshooting, PROLOG

Source: Russell & Norvig 4th Ed, Ch 9, p. 315-335

Introduction to Expert Systems

Definition and Core Characteristics

An **Expert System** is an intelligent computer program that uses knowledge and inference procedures to solve complex problems that typically require human expertise.

Architecture of an Expert System

An Expert System consists of six principal components:

1. **Knowledge Base:** Stores domain-specific knowledge (rules, facts, heuristics).
2. **Inference Engine:** The brain; executes forward/backward chaining to draw conclusions.
3. **Working Memory (Global Database):** Stores current state, input facts, and temporary conclusions.
4. **User Interface:** Facilitates interactive dialogue between non-expert users and the system.
5. **Explanation Facility:** Explains the reasoning behind conclusions (“Why” and “How” questions).
6. **Knowledge Acquisition Module:** Allows domain experts to update rules without rewriting code.

Expert System Architecture

User ↔ User Interface ↔ Inference Engine ↔ Knowledge Base

Inference Engine ↔ Working Memory + Explanation Facility

Source: Russell & Norvig 4th Ed, Ch 1, p. 40-42